

Identify Contrails

Intermediate Machine Learning

SoSe23

Kaggle (Group) Competition

Data Structure

Data Explorer

450.91 GB

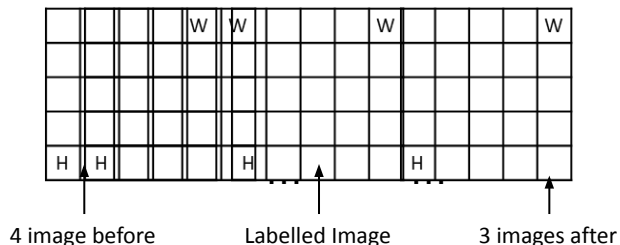
- test
- train
 - 100021648977641407
 - band_08.npy
 - band_09.npy
 - band_10.npy
 - band_11.npy
 - band_12.npy
 - band_13.npy
 - band_14.npy
 - band_15.npy
 - band_16.npy
 - human_individual_r
 - human_pixel_mask
 - 10006035275827755
 - 10006604673592581
 - 100071707854144925
 - 100082372892803174
 - 10009347802007906
 - 10014362066026759
 - 10016536018877742

Train Dataset: 20529 directories with unique record_id
“Each example (record_id) contains exactly one labelled frame.”

Example: record_id =
1000216489776414077

Each band represents an infrared channel at different wavelengths and is converted to brightness temperatures based on the calibration parameters.

Band_xx.npy file: array with size of $H \times W \times T$, where,
 $T = n_times_before + n_times_after + 1$, $\{ T = 4 + 3 + 1 = 8 \}$
representing the number of images in the sequence.
There are n_times_before (4) and n_times_after (3) images before and after the labelled frame respectively.



Band_xx file

ABI Band No.	Approx. Central Wavelength (μm)	Band "Nickname"	Band Type
8	6.2	"Upper-Level Tropospheric Water Vapor" Band	IR
9	6.9	"Mid-Level Tropospheric Water Vapor" Band	IR
10	7.3	"Lower-level Water Vapor" Band	IR
11	8.4	"Cloud-Top Phase" Band	IR
12	9.6	"Ozone Band"	IR
13	10.3	"Clean" IR Longwave Window Band	IR
14	11.2	IR Longwave Window Band	IR
15	12.3	"Dirty" Longwave Window Band	IR
16	13.3	"CO ₂ " longwave infrared	IR

Compilation of RGB Recipes

How to create the standard RGB images from METEOSAT/SEVIRI and MetOp/AVHRR and VIIRS data?

Ash RGB

Colour beam	Channel (difference)	Range			Gamma
Red	IR12.0 - IR10.8	-4	+2	K	1
Green	IR10.8 - IR8.7	-4	+5	K	1
Blue	IR10.8	243	303	K	1

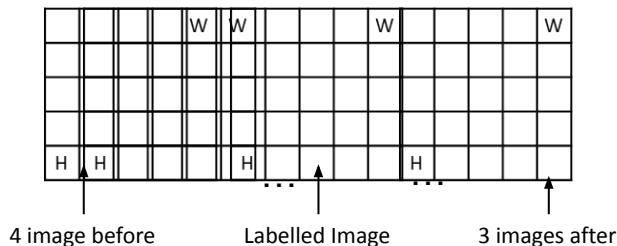
Other files in the Data Structure

Data Explorer

450.91 GB

- ▶ test
- ▼ train
 - ▼ 100021648977641407
 - band_08.npy
 - band_09.npy
 - band_10.npy
 - band_11.npy
 - band_12.npy
 - band_13.npy
 - band_14.npy
 - band_15.npy
 - band_16.npy
 - human_individual_r
 - human_pixel_mask
 - ▶ 10006035275827755
 - ▶ 10006604673592581
 - ▶ 100071707854144925
 - ▶ 100082372892803171
 - ▶ 10009347802007906
 - ▶ 10014362066026759
 - ▶ 10016536018877742

Numpy array with size of $H \times W \times 1 \times R$,
Each example is labelled by R individual human labellers.
The labelled masks have value either 0 or 1 and correspond to the 5th image (labelled) in band_xx.npy



Array with size of $H \times W \times 1$ containing the binary ground truth.
A pixel is regarded as contrail pixel in evaluation if it is labelled as contrail by $>R/2$ labellers.

Visualizing Data

Ash RGB

```
_T11_BOUNDS = (243, 303)
_CLOUD_TOP_TDIFF_BOUNDS = (-4, 5)
_TDIFF_BOUNDS = (-4, 2)

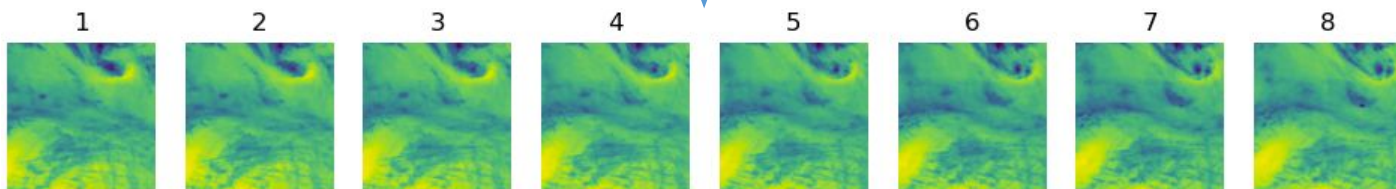
def normalize_range(data, bounds):
    """Maps data to the range [0, 1]."""
    return (data - bounds[0]) / (bounds[1] - bounds[0])

r = normalize_range(band15 - band14, _TDIFF_BOUNDS)
g = normalize_range(band14 - band11, _CLOUD_TOP_TDIFF_BOUNDS)
b = normalize_range(band14, _T11_BOUNDS)
false_color = np.clip(np.stack([r, g, b], axis=2), 0, 1)
```

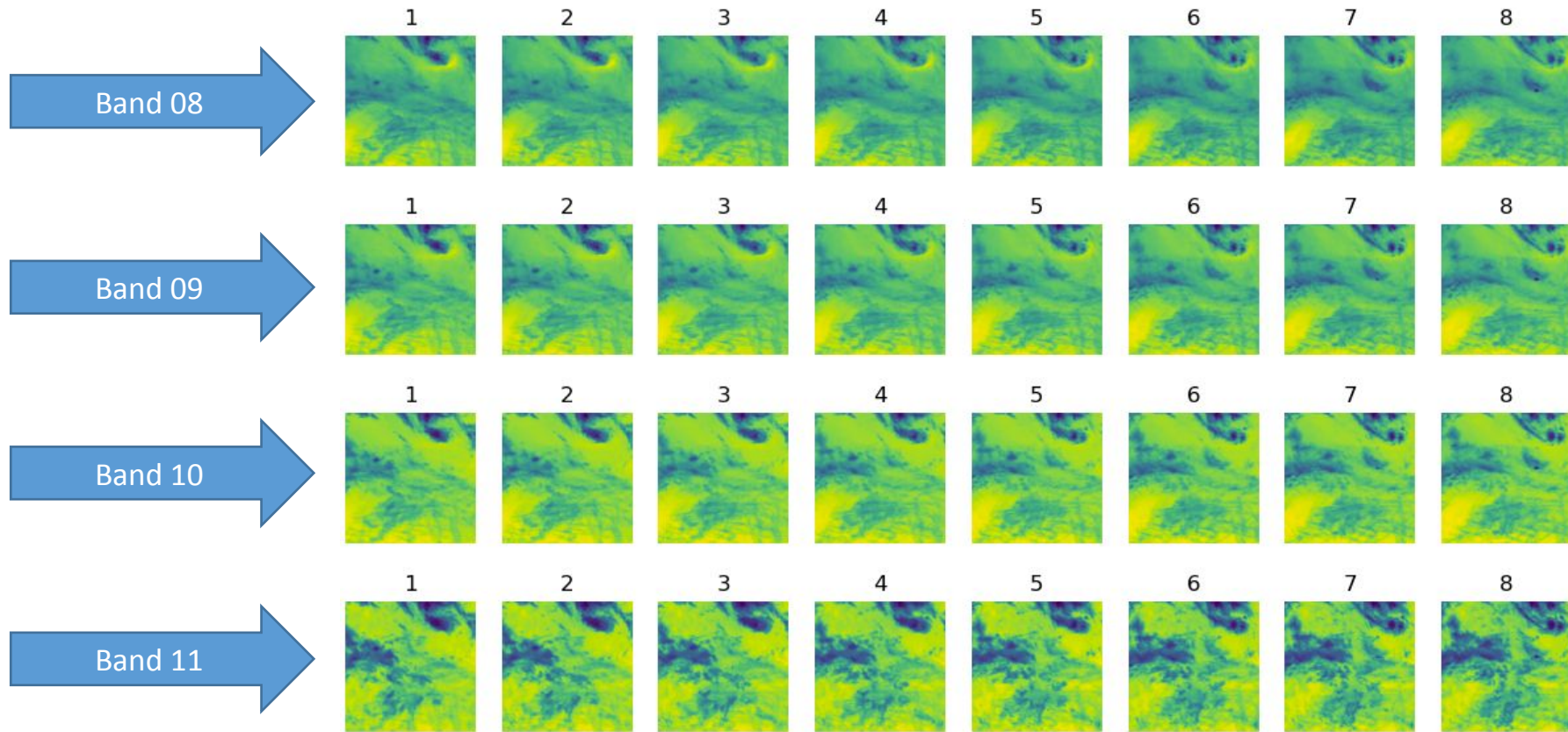
Colour beam	Channel (difference)	Range			Gamma
Red	IR12.0 - IR10.8	-4	+2	K	1
Green	IR10.8 - IR8.7	-4	+5	K	1
Blue	IR10.8	243	303	K	1

```
plt.figure(figsize=(12, 12))
for i in range(8):
    img_test = band8[... , i]
    ax = plt.subplot(1, 8, i+1, title = i+1)
    ax.imshow(img_test)
    ax.axis('off')
```

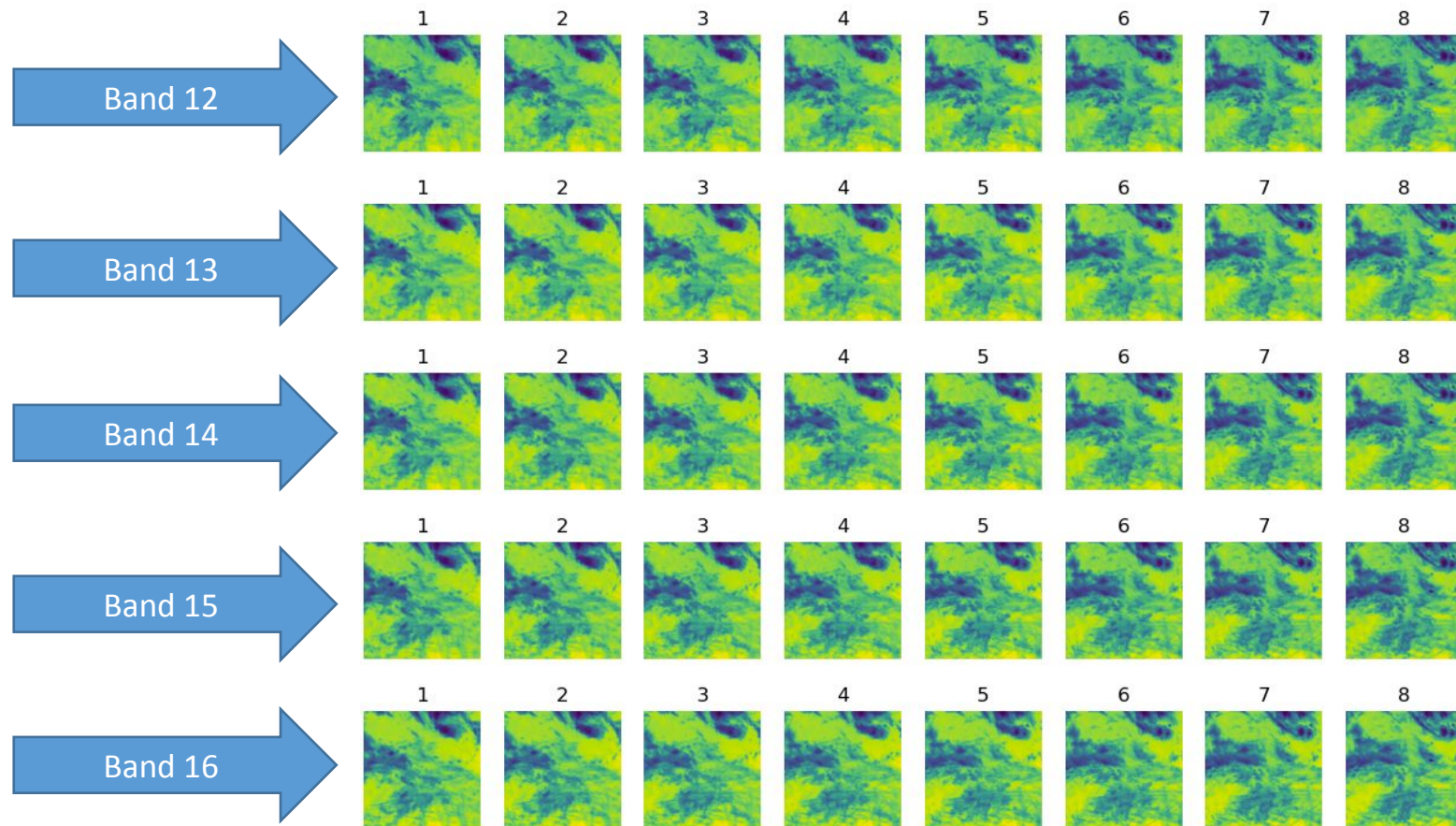
Band
08



Band 08 to Band 11..... (out of Band 16)



...Band 12 to Band 16 (out of Band 16)



Inference from Visualization

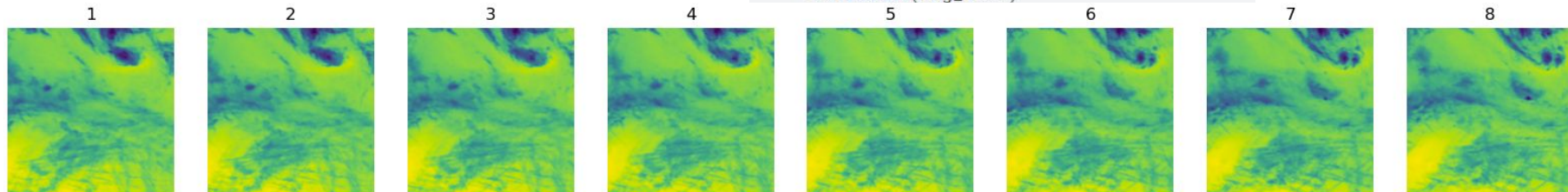
- Band 08, Band 09 and Band 10: provide the clearest imagery of Contrail patterns, while already filtering out CO₂, Clouds, etc.
- This is also evident through the ABI BANDS QUICK INFORMATION GUIDES at <https://www.goes-r.gov/mission/ABI-bands-quick-info.html>, which is summarized as,

ABI Band No.	Approx. Central Wavelength (μm)	Band "Nickname"	Band Type
8	6.2	"Upper-Level Tropospheric Water Vapor" Band	IR
9	6.9	"Mid-Level Tropospheric Water Vapor" Band	IR
10	7.3	"Lower-level Water Vapor" Band	IR

- Adding effects of all 3 bands, we can visualize them as

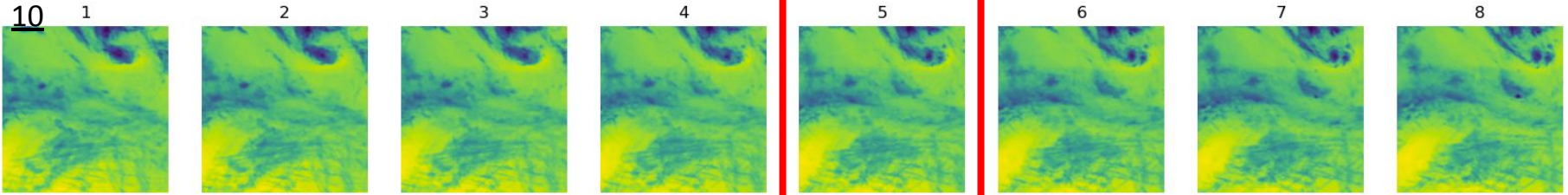
```
trial_band = band8 + band9 + band10
print(trial_band.shape)

plt.figure(figsize=(20, 20))
for i in range(8):
    img_test = trial_band[:, :, i]
    ax = plt.subplot(1, 8, i+1, title = i+1)
    ax.imshow(img_test)
```



Using Training Data meaningfully

Band 08 + Band 09 + Band

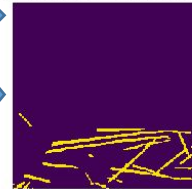


Can be
perceived as
time-series of
images???

Labelled
Image

Ground truth contrail mask

- Contrails must contain at least 10 pixels
- At some time in their life, Contrails must be at least 3x longer than they are wide
- Contrails must either appear suddenly or enter from the sides of the image
- Contrails should be visible in at least two image



Linear structures of Contrails in images can be identified through series of Conv & MaxPool layers.

Data Pre-processing

Understanding data:

- Unique record_id
- 9 bands
 - Each band having 8 images
- 1 labelled image

Preparing data

- Extract individual image
- Apply mathematical manipulations to selected image
 - Delta Method:
 - Spectral Image Analysis:
- Configure DataLoader to be readily used by Model design team

Conducting Spectral Index Processing

- Bands of Interest: Band 13, Band 14

- Theory:

```
# calculate spectral index  
spectral_index = np.log(bandA/bandB) / np.log(lam[band_numberA]/lam[band_numberB])
```

```
(spectral_map - s_min) / (s_max - s_min)
```

- Intricate logarithmic application on mathematical derivation of any 2 given bands
- Further normalization

Spectral index approach

Motivation

- Definition (pixel-wise):

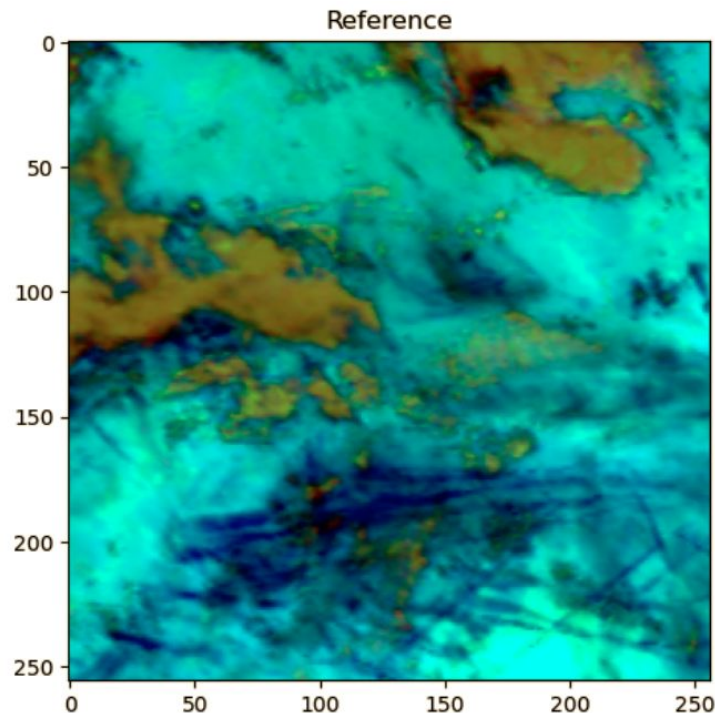
$$\alpha = \frac{\partial \log I_{\lambda}}{\partial \log \lambda} \approx \frac{\Delta \log I}{\Delta \log \lambda}$$

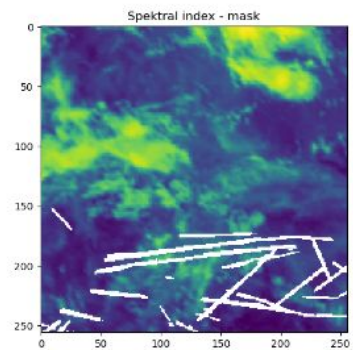
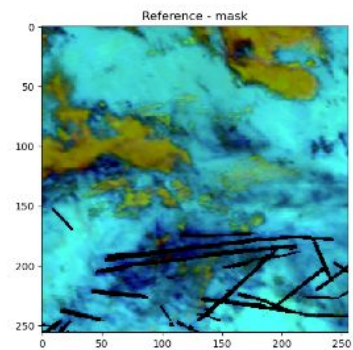
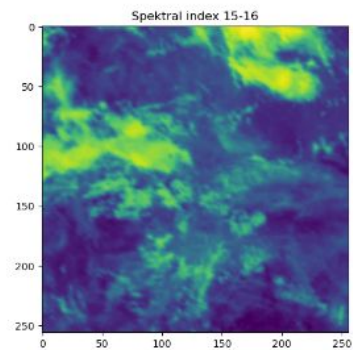
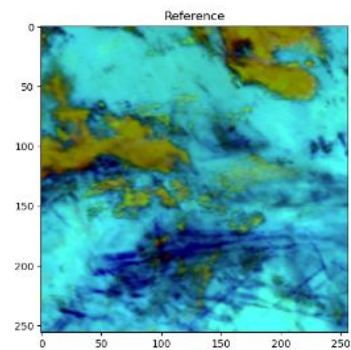
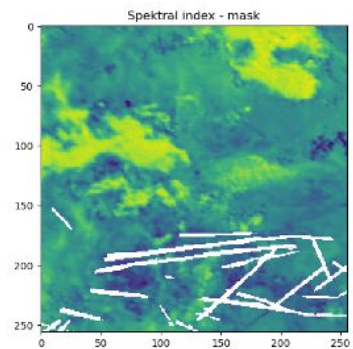
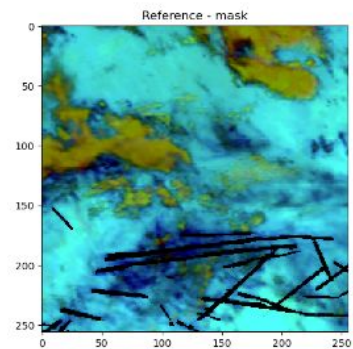
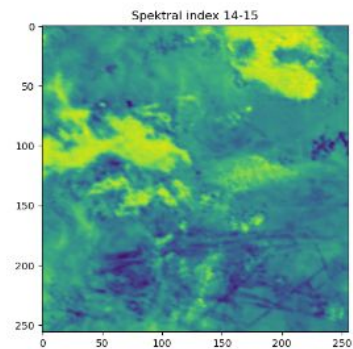
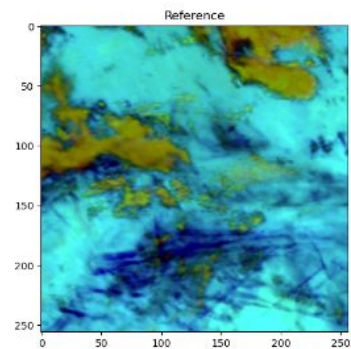
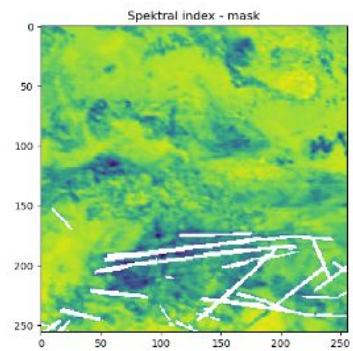
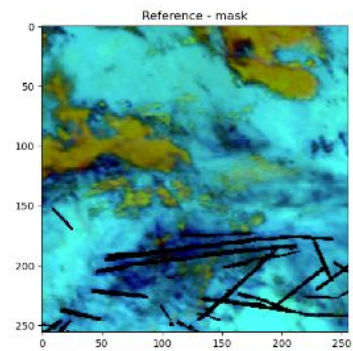
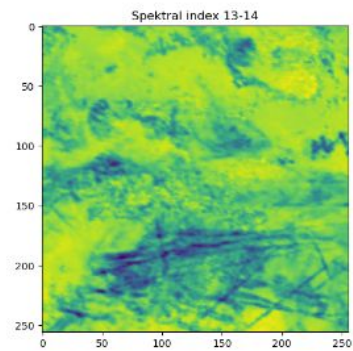
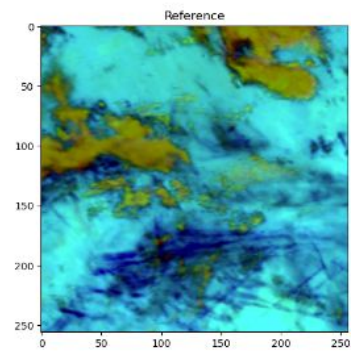
- Interpretation:

- Extraction of spectral features
- Independent of (pixel) brightness
- Particle size and composition $\Rightarrow$ specific α -value

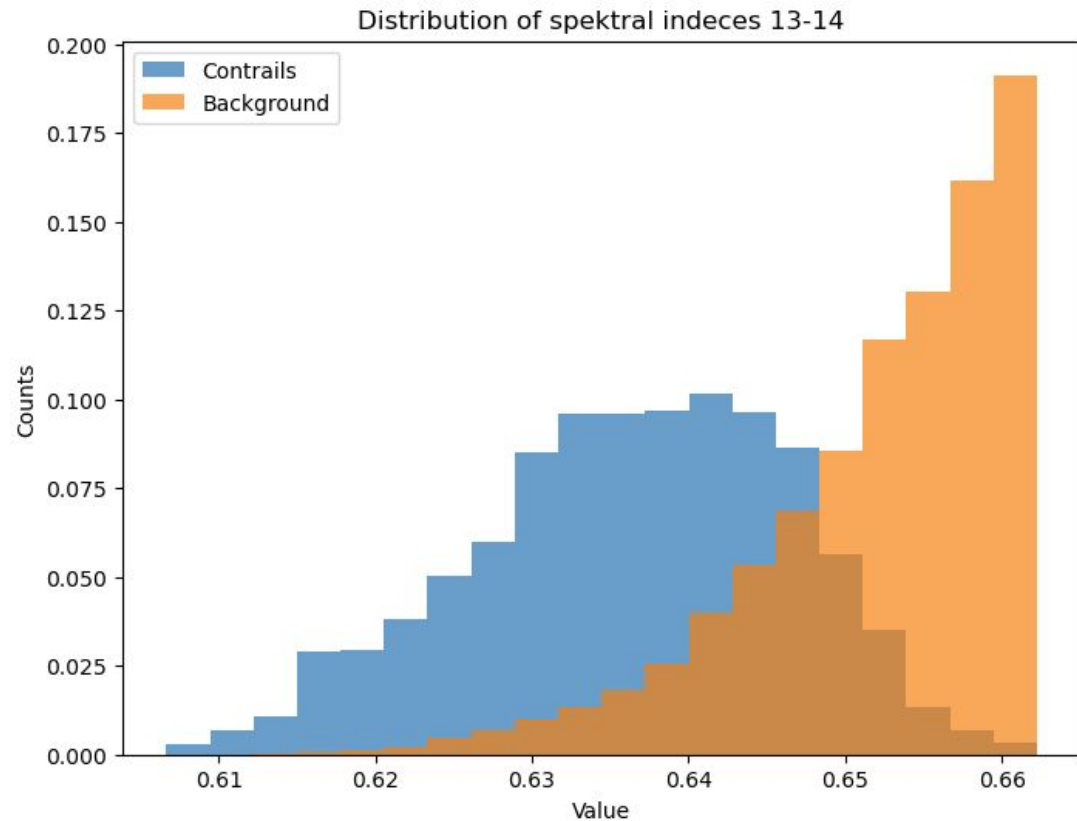
- Goals:

- Find bands of highest contrails/background contrast
- Surpass reference false color images

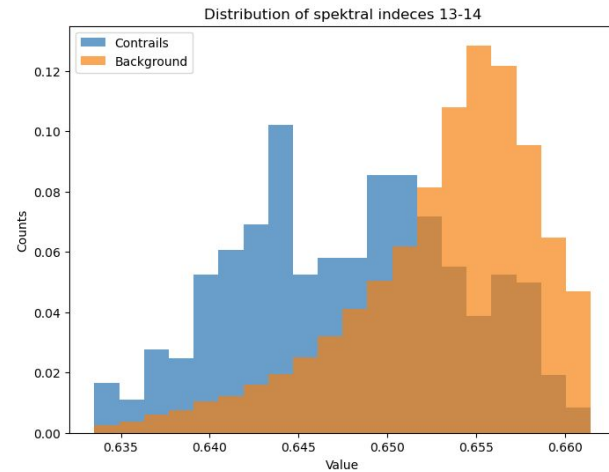




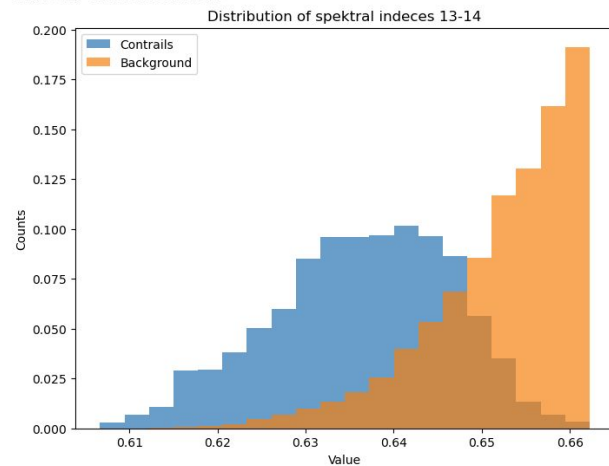
means: 0.6364373294996907 0.6062901304516296
Cor(13-14) -0.29539306441813673



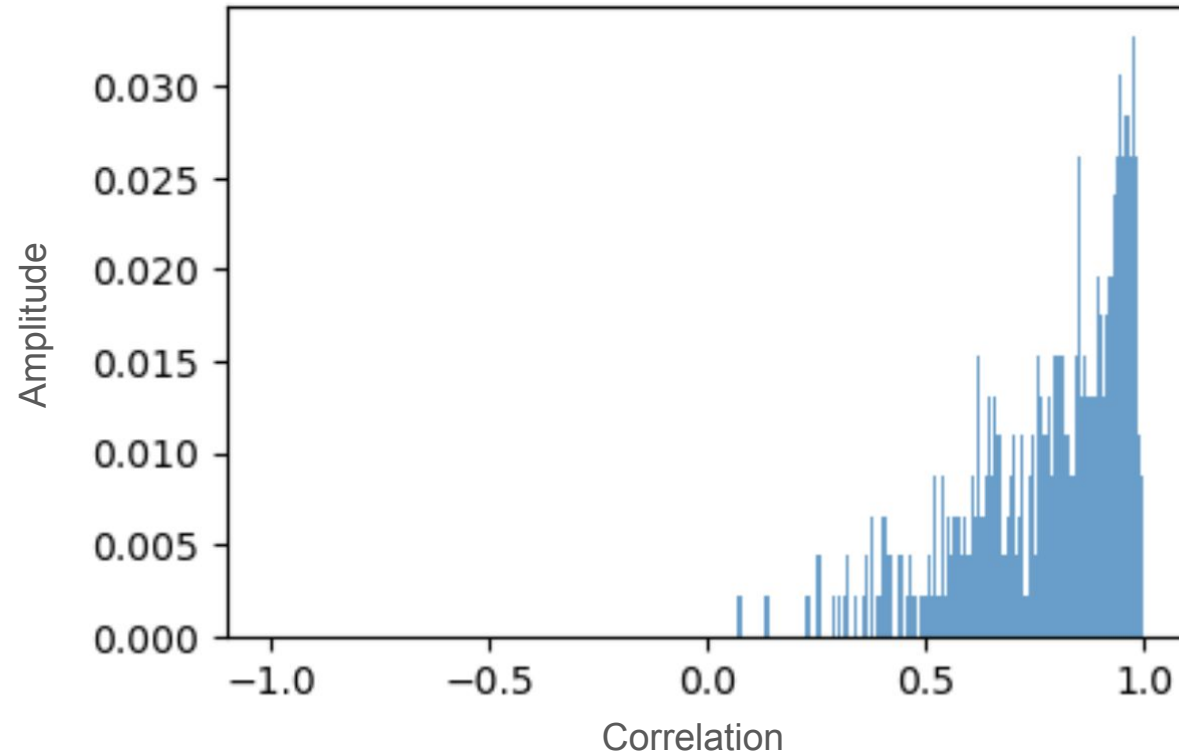
means: 0.6478298416259131 0.6109579538288987
Cor(13-14) 0.14393087865832047



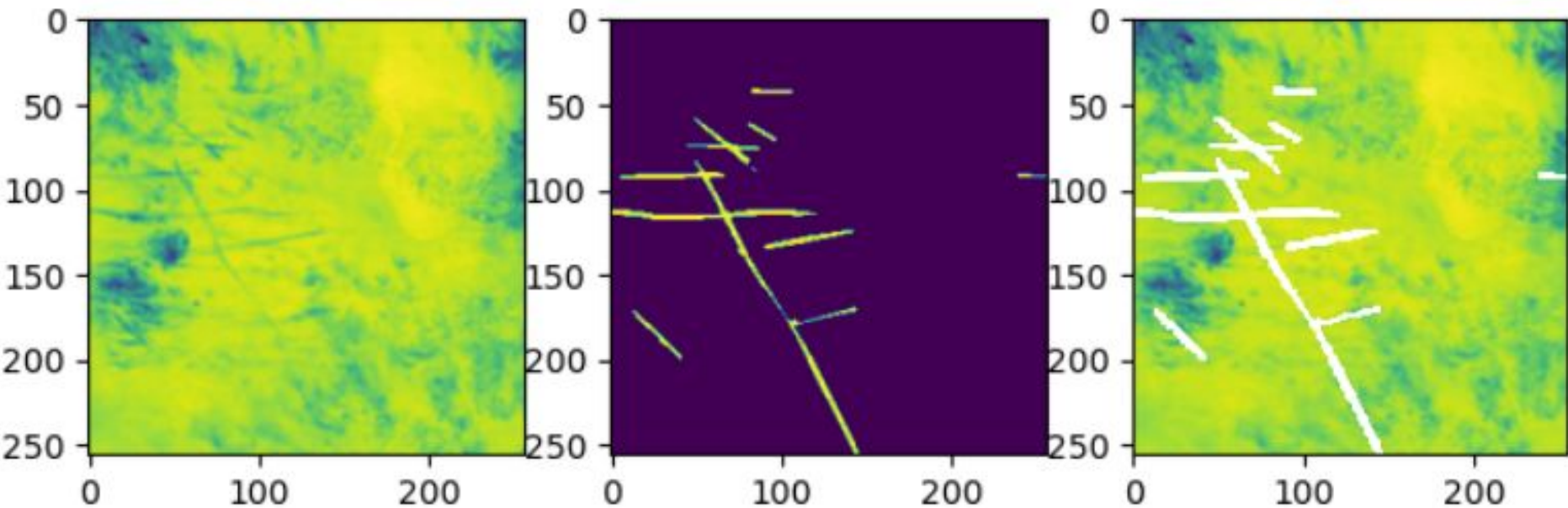
means: 0.6364373294996907 0.6062901304516296
Cor(13-14) -0.29539306441813673



Correlation distribution



Why are correlations so high?



Code snippets

```
def load_band(band_number, record_id = example_record_id, **kwargs):  
    # access base dir  
    BASE_DIR = kwargs["BASE_DIR"]  
  
    # load band  
    with open(os.path.join(BASE_DIR, record_id, f'band_{band_number:02}.npz'), 'rb') as f:  
        band_i = np.load(f)  
  
    # return image sequence  
    return band_i
```

Load band data as
Numpy Array

```
def base_spectral_index(band_numberA, band_numberB, record_id = example_record_id, limit_frame = True,  
                        e, **kwargs):  
    # get wavelengths  
    lam = kwargs["lambda"]  
  
    # individual bands  
    bandA = load_band(band_numberA, record_id=record_id, **kwargs)  
    bandB = load_band(band_numberB, record_id=record_id, **kwargs)  
  
    # calculate spectral index  
    spectral_index = np.log(bandA/bandB) / np.log(lam[band_numberA]/lam[band_numberB])  
  
    if limit_frame:  
        # get just one frame of interest  
        spectral_index = spectral_index[..., kwargs["N_TIMES_BEFORE"]]  
  
    # return the spectral index  
    return spectral_index
```

Calculate Spectral
Index

Calculate Range
for Normalization

```
def normalize_range(spectral_map, band_numberA, band_numberB, **kwargs):  
    """Maps spectral_map to the range [0, 1]."""  
    # get bounds  
    s_min, s_max = kwargs["limits_spectralIndex"][(min(band_numberA, band_numberB), max(band_numberA,  
band_numberB))]  
  
    # normalize  
    return (spectral_map - s_min) / (s_max - s_min)
```

Code snippets (2)

```
def spektral_index(band_numberA, band_numberB, record_id = example_record_id, limit_frame = False, *  
**kwargs):  
    # get spectral index map  
    s_map = base_spektral_index(band_numberA, band_numberB, record_id = record_id, limit_frame = lim  
it_frame, **kwargs)
```

```
    # normalize it  
    s_map = normalize_range(s_map, band_numberA, band_numberB, **kwargs)  
  
    # clip it  
    s_map = np.clip(s_map, 0, 1)  
  
    return s_map
```

Single function for
passing Bands, as
parameters

```
def load_spektral_index(record_id = example_record_id, limit_frame = False, **kwargs):  
    # get spectral index map  
    s_map = base_spektral_index(band_numberA, band_numberB, record_id = record_id, limit_frame = lim  
it_frame, **kwargs)
```

```
    # normalize it  
    s_map = normalize_range(s_map, band_numberA, band_numberB, **kwargs)  
  
    # clip it  
    s_map = np.clip(s_map, 0, 1)  
  
    # get mask  
    mask = get_human_mask(record_id = record_id, **kwargs)  
  
    return s_map, mask
```

Return
Normalized
Spectral Index

```
def get_human_mask(record_id = example_record_id, **kwargs):  
    # access base dir  
    BASE_DIR = kwargs["BASE_DIR"]  
  
    # load ground truth mask  
    with open(os.path.join(BASE_DIR, record_id, 'human_pixel_masks.npy'), 'rb') as f:  
        human_pixel_mask = np.load(f)  
  
    # return human pixel mask as numpy array  
    return human_pixel_mask
```

Return Human
Mask (label)

Code snippets (3)

```
class ContrailDataset_SI(Dataset):
    def __init__(self, base_dir, data_type='train', transform=None):
        assert data_type in ['train', 'validation', 'test'], \
            "'data_type' should be one of 'train', 'validation', or 'test'"

        self.base_dir = base_dir
        self.data_type = data_type
        self.transform = transform
        self.record = os.listdir(self.base_dir + '/' + self.data_type)

    def __len__(self):
        return len(self.record)

    def __getitem__(self, idx):
        record_id = self.record[idx]
        record_dir = os.path.join(self.base_dir, self.data_type, record_id)

        spectralIndex_map, pixel_masks = load_spektral_index(record_id = record_id,
            band_numberA = 13,
            band_numberB = 14,
            limit_frame = False,
            **baseSettings_SpectralIndex)
```

Defining Class for Dataset, incl.
Mathematical manipulation

Bands of
Interest

Std Dataloader function

```
def get_dataloader_SI(base_dir, data_type, batch_size, transform=None):
    dataset = ContrailDataset_SI(base_dir, data_type=data_type, transform=transform)
    dataloader = DataLoader(dataset, batch_size=batch_size, collate_fn=collate_fn)

    return dataloader
```

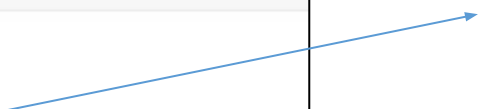
Instantiating DataLoader

```
#Instantiate the dataloader by defining the global path and the  
train_dataloader_Del = get_dataloader(GLOBAL_PATH, 'train', batch_size=BATCH_SIZE)  
validation_dataloader_Del = get_dataloader(GLOBAL_PATH, 'validation', batch_size=BATCH_SIZE)
```

```
bands_SI, masks_SI = next(iter(train_dataloader_SI))  
print(bands_SI.shape) # Batch size, H, W, Timeframe  
print(masks_SI.shape)
```

```
torch.Size([128, 256, 256, 8])  
torch.Size([128, 256, 256, 1])
```

Ready to pass onto model team



Conclusion

- Highest visual contrast:
 - Spectral index for bands 13-14
- Issue:
 - Occasionally high correlation Background-to-Contrail
- Possible reasons:
 - Broader regions in human masks than visually detectable
 - Not all contrails appear in human masks
- Data generator provided to the Model-Group 